# Lab 101: Linux Hardening for SOC Analysts

Course: Blue Team SOC Foundations
Module: Automated Log Hardening & Service Reduction
Date: June 24, 2026
Prerequisites: Ubuntu 24.04, GitHub Account, Basic CLI Knowledge

---

## Lab Overview

In this lab, you will develop a Bash script to automate the hardening of an Ubuntu system based on CIS (Center for Internet Security) Benchmark principles. You will focus on reducing the attack surface by disabling non-essential services, configuring a host-based firewall (UFW), and implementing robust logging. This mirrors the initial setup required for a secure SOC analyst workstation or a production server. .

**Key Learning Objectives:**

- Implement Bash Strict Mode (set -euo pipefail) for safe automation.

- Audit and whitelist essential system services.

- Configure UFW with a "Default Deny" policy.

- Create an immutable audit trail of system changes.

***Safety Note:***
*Console Access: Ensure you have physical or console access to the machine before running firewall or SSH hardening scripts. Misconfiguration can lock you out of remote sessions.*

*Backup: Always snapshot your virtual machine or backup critical data before running hardening scripts.*

Dry Run: Test scripts in a non-production environment first. The script provided includes safety checks, but verification is your responsibility.

---

## Phase 1: Script Initialization & Logging

Step 1: Create the Script File
Create a new file named harden_ubuntu.sh. This script will serve as your primary tool for enforcing security policies.

**Task**:
nano harden_ubuntu.sh

Step 2: Implement Strict Mode & Logging
Paste the following code into the file. This sets up the "Unofficial Bash Strict Mode" to prevent silent failures and defines a logging function to track every action taken by the script.

```
#!/bin/bash
# SOC Lab: Ubuntu Hardening Script
# Author: [Your Name]
# CIS Benchmark Level: 1
```

```
# 1. Strict Mode: Exit on error, unset variable, or pipe failureset -euo pipefail
# 2. Configuration
AUDIT_LOG="/var/log/hardening_audit.log"
SCRIPT_NAME="HardeningScript"
# 3. Logging Functionlog_action() {
   local message="$1"
   # Log to system syslog
   logger -t "$SCRIPT_NAME" "$message"
   # Log to local file with timestamp
   echo "$(date '+%F %T') - $message" | sudo tee -a "$AUDIT_LOG" > /dev/null
}
# 4. Root Checkif [[ $EUID -ne 0 ]]; then
     echo "This script must be run as root (use sudo)"
     exit 1fi
# 5. Execution Start
log_action "Script execution started."
```

Save and Exit: Ctrl+O, Enter, Ctrl+X.

Step 3: Verify Syntax

Before proceeding, ensure the script has no syntax errors.

**Task**:

```
chmod 700 harden_ubuntu.sh
bash -n harden_ubuntu.sh
```

Success Criteria: chmod succeeds silently. bash -n returns no output (no errors).

---

## Phase 2: Service Auditing & Reduction

Step 1: Define the Hardening Function

We will create a function that lists all enabled services, compares them against a "whitelist" of essential services, and flags non-essential ones for disabling.

**Task**: Open the script again (nano harden_ubuntu.sh) and append the following function before the main execution flow (at the bottom of the file):

```
# ----------------------------------------------------------# Function: Harden Services# Purpose: Disable non-essential services to reduce attack surface#
----------------------------------------------------------harden_services() {
   log_action "Starting Service Hardening..."

   # Define Allowed Services (Whitelist)
   # Add services you NEED (e.g., ssh, ufw, cron, networking)
   local allowed_services=(
      "ssh" "ufw" "cron" "systemd-journald" "networking"
      "systemd-resolved" "dbus" "rsyslog" "apparmor"
      "NetworkManager" "wpa_supplicant" "avahi-daemon"
      "power-profiles-daemon" "thermald" "udisks2"
      "accounts-daemon" "snapd" "unattended-upgrades"
   )

   # List all enabled services
   # FIX: Added '|| true' to prevent script exit if systemctl returns non-zero
   # FIX: Added 'tail -n +2' to skip the header row ("UNIT")
```

```
    local all_services=$(systemctl list-unit-files --type=service --state=enabled
--no-pager 2>/dev/null | tail -n +2 | awk '{print $1}' | sed 's/.service//' || true)

    # Safety check: If the list is empty, stop to avoid disabling everything
    if [[ -z "$all_services" ]]; then
        log_action "ERROR: Service list is empty. Skipping service hardening."
        return 1
    fi

    for service in $all_services; do
        # Skip empty lines or entries that don't start with a letter
        if [[ ! "$service" =~ ^[a-zA-Z] ]]; then
            continue
        fi

        # Check if service is in the allowed list
        if [[ ! " ${allowed_services[@]} " =~ " ${service} " ]]; then
            echo "Found non-essential service: $service"
            # SAFETY: Command is commented out for the lab. Remove '#' to execute.
            # systemctl disable --now "$service"
            log_action "FLAGGED for disable: $service"
        fi
    done

    log_action "Service Hardening complete."
}
```

Step 2: Update Main Execution Flow
Ensure the function is called at the end of the script. Add the following line to the very bottom of harden_ubuntu.sh:
harden_services

Step 3: Test the Service Audit
Run the script to see which services would be flagged.

**Task**:
sudo ./harden_ubuntu.sh
Expected Output:

A list of "Found non-essential service: [name]"
(e.g., cups, bluetooth, ModemManager).

No script crashes.

Check the log: cat /var/log/hardening_audit.log.

---

## Phase 3: Firewall Automation (UFW)

Step 1: Add Firewall Function
We will configure UFW to deny all incoming traffic by default and only allow essential services (SSH).

**Task**: Append this function to harden_ubuntu.sh (before the main execution calls):

```
# ------------------------------------------------------# Function: Harden Firewall (UFW)# Purpose: Configure default deny policy#
------------------------------------------------------harden_firewall() {
    log_action "Starting Firewall Hardening..."

    # 1. Reset UFW to clean state
    ufw --force reset > /dev/null
    log_action "UFW reset complete."

    # 2. Set Defaults
    ufw default deny incoming > /dev/null
    ufw default allow outgoing > /dev/null
    log_action "UFW defaults set: Deny Incoming, Allow Outgoing."

    # 3. Allow Essential Services
    ufw allow ssh comment 'Allow SSH for Admin' > /dev/null
    ufw allow in on lo > /dev/null
    ufw allow out on lo > /dev/null

    # 4. Enable Firewall
    # SAFETY: Uncomment the line below only when you are sure SSH is allowed
    echo "Ready to enable UFW. SSH is allowed."
    # ufw --force enable

    log_action "Firewall Hardening complete (Pending Enable)."
}
```

Step 2: Update Main Execution Flow
Add the call to harden_firewall at the bottom of the script, after harden_services.

```
harden_services
harden_firewall
```

Step 3: Test Firewall Configuration
Run the script again. It will reset UFW and prepare the rules without enabling them (safety first).

**Task**:
sudo ./harden_ubuntu.shsudo ufw status verbose
Expected Output:

Status: inactive (since we didn't uncomment the enable line).

Rules listed: Allow ssh, Deny (incoming), Allow outgoing.

---

## Phase 4: Final Cleanup & Documentation

Step 1: Remove Debug Output
If you added any echo "DEBUG..." lines during testing, remove them now to ensure clean logs in production.

Step 2: Final Commit
Commit your finalized script to your GitHub repository.

**Task**:
git add harden_ubuntu.sh

```
git commit -m "feat: Finalize service hardening logic (CIS 2.x) and UFW defaults (CIS 3.x)

- Fixed parsing error: Added 'tail -n +2' to skip systemctl header
- Fixed strict mode crash: Added '|| true' to pipeline
- Expanded whitelist: Added apparmor, rsyslog, NetworkManager, snapd
- Removed debug output for production readiness
"
git push origin main
```

---

*When you are ready, proceed to the Lab 102: Network Traffic Analysis with Suricata guide.*